ABSTRACT

The primary cause of death globally is cardiovascular disease (CVD), making timely and accurate diagnosis crucial for preventing further complications. Researchers are turning to machine learning (ML) to aid in CVD diagnosis, as ML algorithms can uncover patterns and relationships within medical data to detect or predict disease progression. However, predicting CVD presents challenges due to the complexity of healthcare datasets, making it difficult for practitioners to accurately predict and understand feature relationships. Feature selection, therefore, plays a vital role in optimizing ML algorithm performance by identifying key features within the dataset. To address this, a study introduces the Automated Cardiovascular Disease Diagnosis using Honey Badger Optimization with Modified Deep Learning (ACVD-HBOMDL) Model. This approach focuses on CVD classification through feature selection ad hyperparameter tuning. Initially, the ACVD-HBOMDL method preprocesses medical data using min-max scaling. It then employs the Honey Badger Optimization (HBO) algorithm to select an optimal subset of features. For CVD classification, a modified deep learning neural network (DLMNN) classifier is utilized, with Bayesian optimization used to optimally select its hyperparameters. Experimental results of the ACVD-HBOMDL technique are evaluated on benchmark medical datasets, demonstrating its superiority over existing techniques.

CHAPTER 1: INTRODUCTION

Cardiovascular disease (CVD) stands out as a significant health issue impacting the heart and blood vessels, often resulting in loss of life due to delayed diagnosis. Early detection is crucial, and preemptive risk assessment could greatly alleviate this challenge. To address this, leveraging patient data alongside artificial intelligence (AI) and domain expertise is paramount in healthcare. Traditional manual analysis of heart disease data is both time-intensive and prone to misdiagnosis. Thus, AI, particularly deep learning (DL) and machine learning (ML), has emerged as a prevalent approach for predictive analytics in this domain. However, CVD detection remains complex, involving the examination of numerous features and the selection of optimal models. Feature selection (FS) and hyperparameter tuning are pivotal stages in enhancing CVD detection methods. Implementing these techniques not only improves accuracy but also mitigates overfitting and bolsters generalization performance. Consequently, we present an Automated Cardiovascular Disease Diagnosis system utilizing an Optimization technique with a Modified Deep Learning model, aimed at advancing the efficiency and efficacy of CVD detection.

1.1. OBJECTIVE

Cardiovascular disease (CVD) remains a global health threat, often leading to fatalities due to delayed diagnosis. Early detection is crucial, and leveraging patient data with artificial intelligence (AI) offers a promising solution. While traditional methods for analyzing heart disease data are time-consuming and prone to errors, AI techniques, particularly deep learning (DL) and machine learning (ML), have emerged as powerful tools for predictive analytics in this domain.

However, accurately detecting CVD remains complex. Existing limitations include the need for comprehensive feature analysis, optimal model selection, and user-friendly interfaces for real-world application.

This project addresses these challenges by introducing ACVD-HBOMDL, an Automated Cardiovascular Disease Diagnosis system utilizing a novel combination of techniques:

1. **Enhanced Feature Selection:** ACVD-HBOMDL tackles limitations in traditional feature selection by employing a "honey badger optimization" technique. This method goes beyond standard approaches to identify the most informative features from patient data, leading to a more accurate diagnosis.
2. **Stacking Classifier for Improved Accuracy:** ACVD-HBOMDL incorporates a "stacking classifier," a powerful ensemble learning technique. By combining multiple models, this approach achieves a higher level of accuracy in CVD classification compared to standalone models.
3. **Deep Learning and Bayesian Optimization:** ACVD-HBOMDL leverages a modified deep learning model specifically designed for CVD diagnosis. This model is further optimized using Bayesian optimization, a technique that refines the model's performance by learning from past iterations.
4. **User-Friendly Interface:** ACVD-HBOMDL prioritizes usability by incorporating a user-friendly Flask front-end with a secure authentication system. This makes the system practical and accessible for real-world application in clinical settings.

By combining these elements, ACVD-HBOMDL strives to deliver a more accurate, efficient, and user-friendly system for diagnosing cardiovascular disease, ultimately contributing to improved patient care and outcomes.

1.2. MOTIVATION

Cardiovascular disease (CVD) remains a silent killer, silently claiming millions of lives globally each year. While advancements have been made, early detection of CVD is still a significant challenge. Traditional methods for analyzing heart disease data can be time-consuming and prone to human error, leading to delayed diagnoses and potentially poorer patient outcomes.

The motivation for this project stems from this very real-world need. We recognize the limitations of current practices and believe that leveraging artificial intelligence (AI) offers a promising solution. By integrating AI techniques like deep learning (DL) and machine learning (ML) with readily available patient data, we aim to:

- **Reduce Missed Diagnoses:** Every missed diagnosis represents a missed opportunity for early intervention. AI's ability to analyze complex data sets and identify subtle patterns can significantly improve diagnostic accuracy, leading to fewer missed cases and potentially saving lives.
- **Streamline Workflows:** Clinicians often face heavy workloads and limited time. Automating critical stages of CVD diagnosis, such as feature selection and hyperparameter tuning for ML models, can free up valuable time for patient care.
- **Optimize Resource Allocation:** More accurate diagnoses can lead to a more efficient allocation of healthcare resources. By identifying true cases of CVD earlier, unnecessary procedures and interventions can be avoided, allowing resources to be focused on those who need them most.

This project is not simply about developing a novel technique, but about creating a practical tool that can make a tangible difference in the fight against CVD. By improving diagnostic accuracy, efficiency, and reducing the risk of human error, we hope to empower clinicians with a powerful tool to combat this critical health issue and ultimately improve patient care.

1.3. BACKGROUND

The project's backdrop is on the growing importance of using machine learning (ML) techniques to the identification and assessment of cardiovascular risk and heart disease (HD). Several ML approaches, such as Naïve Bayes (NB), support vector machines (SVM), extreme gradient boosting (XGBoost), and artificial neural networks (ANN), have been investigated for HD prediction and diagnosis in numerous publications, including those by Nagavelli et al. and Ali et al. These methods improve the precision and effectiveness of disease diagnosis by utilizing characteristics taken from patient data, such as body temperature, electrocardiogram (ECG), and blood oxygenation.

Furthermore, the application of ensemble systems—as suggested by Atallah and Al-Mousa—shows how combining predictions from many ML models can enhance prediction results. Furthermore, it is emphasized how crucial feature selection methods like Relief, Least Absolute Shrinkage Selective Operator, and Minimal Redundancy Maximum Relevance are for improving machine learning models by removing superfluous and unnecessary information.

Against this backdrop, this thesis proposes a novel ACVD-HBOMDL (Automated Cardiovascular Disease Diagnosis using Hybrid Bayesian Optimization and Modified Deep Learning) technique aimed at enhancing the accuracy of cardiovascular disease (CVD) diagnosis. The ACVD-HBOMDL technique integrates data scaling, feature subset selection using hybrid Bayesian optimization (HBO), classification using a modified deep learning model (DLMNN), and parameter optimization using Bayesian optimization (BO). This approach is motivated by the need for more accurate and efficient diagnostic methods in addressing the complexities of CVD detection, ultimately contributing to improved patient care and outcomes.

1.4 PROBLEM STATEMENT

Machine learning algorithms have revolutionized disease prediction by identifying patterns hidden within medical datasets. However, accurately forecasting cardiovascular disease (CVD) remains a significant challenge. The ever-growing size and complexity of healthcare data pose a problem for clinicians struggling to understand feature relationships and make accurate CVD predictions. Traditional feature selection methods often falter in the face of this data explosion.

This project addresses this challenge by introducing the Automated Cardiovascular Disease Diagnosis system with Honey Badger Optimization and Modified Deep Learning Model (ACVD-HBOMDL). ACVD-HBOMDL tackles the feature selection bottleneck by employing a novel "Honey Badger Optimization" technique specifically designed for the intricacies of healthcare data. This technique goes beyond standard methods, identifying the most informative features with greater accuracy, leading to improved CVD prediction. Furthermore, ACVD-HBOMDL integrates a modified Deep Learning model specifically tailored for CVD diagnosis tasks. This model is further optimized using Bayesian optimization, ensuring peak performance. By tackling both feature selection and model optimization, ACVD-HBOMDL aims to unlock the full potential of ML for accurate and efficient CVD diagnosis, ultimately improving patient care and outcomes.

CHAPTER 2: PROJECT OVERVIEW

2.1 PROJECT DESCRIPTION

The ACVD-HBOMDL project aims to revolutionize cardiovascular disease (CVD) diagnosis by integrating cutting-edge machine learning techniques. This comprehensive system employs feature selection and hyperparameter tuning to enhance the accuracy of CVD classification. The process begins with data exploration and image processing, ensuring the efficient handling of medical data. Model generation involves a variety of algorithms, including RepTree, SVM, Bagging, ANN, J48, SC Algorithm, HBOM-DL, and a Stacking Classifier. Users can interact with the system through signup/login and input modules, providing data for prediction. The heart of the system lies in its utilization of min-max scaling for data preprocessing and Honey Badger Optimization (HBO) for feature selection. For classification, a deep learning modified neural network (DLMNN) is employed, with Bayesian optimization utilized to fine-tune hyperparameters. Rigorous testing on benchmark medical datasets will validate the system's efficacy, with the ultimate goal of delivering a reliable, automated solution for early CVD detection, thus advancing patient care and healthcare management practices.

2.2 PROJECT GOALS

This project aims to develop a more accurate and efficient system for diagnosing cardiovascular disease (CVD) by leveraging the power of machine learning (ML) and artificial intelligence (AI). Here are some specific goals:

Improved Diagnostic Accuracy: Develop an AI-powered system (ACVD-HBOMDL) that surpasses traditional methods in accurately identifying CVD cases from patient data.

Enhanced Feature Selection: Implement a novel "Honey Badger Optimization" technique to identify the most informative features within complex healthcare datasets, leading to more accurate predictions.

Optimized Deep Learning Model: Design and integrate a modified deep learning model specifically tailored for CVD diagnosis tasks. Further refine this model using Bayesian optimization to guarantee peak performance.

Streamlined Workflow: Automate critical stages of CVD diagnosis, such as feature selection and hyperparameter tuning for the ML model, freeing up valuable clinician time for patient care.

Increased Efficiency: Enable faster and more efficient diagnoses compared to traditional methods, potentially leading to earlier interventions and improved patient outcomes.

Real-World Usability: Develop a user-friendly interface for ACVD-HBOMDL, potentially using a Flask front-end with secure authentication, to facilitate its practical application in clinical settings.

CHAPTER 3: TECHNICAL SPECIFICATIONS

Table 3.1 Technical Specifications

COMPONENT	SPECIFICATION
Software	**Version/Platform**
Anaconda	Latest
Primary Language	Python
Frontend Framework	Flask
Backend Framework	Jupyter Notebook
Database	Sqlite3
Front-End Technologies	HTML, CSS, JavaScript, Bootstrap4
Hardware Requirements	
Operating System	Windows Only
Processor	i5 and above
RAM	8GB and above
Hard Disk	25 GB (local drive)

CHAPTER 4: THEORETICAL BASIS

ACVD-HBOMDL's success in enhancing cardiovascular disease (CVD) diagnosis hinges on a robust theoretical foundation that draws upon several key concepts from machine learning (ML), deep learning (DL), and optimization techniques.

1. Feature Selection: The Cornerstone of Accurate Diagnosis

The project recognizes the pivotal role of feature selection in crafting effective ML models. Medical datasets often contain a multitude of features, some redundant or irrelevant to CVD diagnosis. Identifying the most informative features – those that hold the key to accurate prediction – is crucial. ACVD-HBOMDL tackles this challenge by:

- **Feature Importance Measures:** Techniques like information gain, correlation analysis, and feature ranking algorithms help quantify the importance of each feature in relation to the target variable (CVD presence or absence). These measures guide the selection process, ensuring the model focuses on the most impactful features.
- **Dimensionality Reduction:** Not only can a smaller sample of useful characteristics increase computing efficiency, but they can also increase model correctness. Principal Component Analysis (PCA) is one technique that can be used to minimize the dimensionality of the data while maintaining the most important information for the categorization of CVDs.

2. Honey Badger Optimization: A Novel Approach to Feature Hunting

ACVD-HBOMDL introduces the innovative Honey Badger Optimization (HBO) technique for feature selection. HBO draws inspiration from the foraging behavior of honey badgers, known for their tenacity and resourcefulness. It falls under the umbrella of swarm intelligence algorithms, which mimic collective behaviors observed in nature to solve complex optimization problems.

Here's how HBO works in ACVD-HBOMDL:

- **Honey Badgers as Feature Hunters:** Each honey badger in the swarm represents a potential feature subset. The "fitness" of each badger is evaluated based on a pre-defined metric, such as the model's accuracy on a validation dataset.
- **Foraging and Exploration:** Honey badgers explore the feature space, searching for high-performing feature combinations. This exploration process can involve techniques like mutation and crossover, inspired by biological evolution.
- **Collaboration and Exploitation:** Successful badgers (those with high-performing feature subsets) share information with others, guiding the swarm towards promising areas of the feature space. This collaboration helps to exploit areas with high potential for accurate CVD detection.

3. Deep Learning: Unveiling Complex Patterns in Medical Data

ACVD-HBOMDL incorporates a modified deep learning model, specifically designed for CVD diagnosis. Deep learning models are a type of artificial neural network (ANN) with multiple layers. Inspired by the human brain, these models excel at uncovering intricate patterns within complex datasets. Here's why deep learning is well-suited for ACVD-HBOMDL:

- **Automatic Feature Extraction:** Deep learning models can automatically learn relevant features directly from the data. This reduces the reliance on handcrafted feature selection and allows the model to identify subtle patterns that might be missed by traditional methods.

- **Non-linear Relationships:** Deep learning models can effectively capture non-linear relationships between features, which are often present in medical data. This capability is crucial for accurately modeling the complex interplay of factors that contribute to CVD development.
- **Model Generalizability:** By learning from a large and diverse dataset of medical data, deep learning models can be trained to generalize well to unseen data, leading to more robust CVD predictions on new patients.

4. Bayesian Optimization: Fine-Tuning the Deep Learning Engine

ACVD-HBOMDL employs Bayesian optimization to refine the hyperparameters of the deep learning model. Hyperparameters are settings that control the learning process of the model, such as the number of layers, learning rate, and activation functions. Finding the optimal hyperparameter configuration is crucial for maximizing model performance.

Here's how Bayesian optimization helps:

- **Probabilistic Approach:** This technique utilizes a probabilistic framework to efficiently search the space of possible hyperparameter combinations. It prioritizes settings with a higher probability of leading to good performance, based on the model's performance with previous configurations.
- **Reduced Evaluation Cost:** By focusing on promising areas of the hyperparameter space, Bayesian optimization reduces the need to evaluate a vast number of configurations, saving time and computational resources.
- **Continuous Learning:** As the optimization process progresses, the model's performance with different hyperparameter settings is fed back into the Bayesian framework. This continuous learning allows the optimizer to refine its search strategy and identify the best possible hyperparameter configuration for accurate CVD diagnosis.

5. Ensemble Learning :

- **Stacking Classifier :** The project description mentions the potential use of a stacking classifier, which is an ensemble learning technique. Ensemble learning combines predictions from multiple models to potentially achieve higher accuracy than any single model. Here, a stacking classifier would combine the predictions from several base models to arrive at a more robust final prediction for CVD diagnosis.

  These theoretical foundations provide the groundwork for ACVD-HBOMDL's ability to analyze complex medical data, identify critical features, and leverage powerful deep learning models to achieve a more accurate and efficient system for diagnosing cardiovascular disease.

CHAPTER 5: DESIGN APPROACH AND DETAILS

5.1 DESIGN APPROACH

5.1.1 HONEY BADGER OPTIMIZATION

Honey Badger Optimization (HBO) is a bio-inspired algorithm designed to tackle feature selection problems. It mimics the foraging behavior of honey badgers and their interactions to efficiently identify the most informative features from a dataset. Here's a breakdown of the design approach:

Classification Model Training: For each honey badger (feature subset), a classification model (e.g., Logistic Regression, Support Vector Machine) is trained on the training data using only the selected features.

Performance Metric: The fitness of a honey badger is evaluated based on the performance of the corresponding classification model on a validation dataset. Common metrics include accuracy, F1-score, or AUC-ROC. This metric guides the optimization process towards selecting features that lead to the most accurate classification for the target variable (e.g., presence/absence of CVD).

Information Sharing: Top-performing honey badgers can share a portion of their selected features with others, guiding them towards potentially better combinations. This leverages the knowledge of successful individuals to improve the overall search process.

Adaptability: The specific foraging and collaboration mechanisms can be adapted or modified based on the specific characteristics of the dataset. This allows for customization based on the problem at hand.

5.2 MATERIALS AND METHODS

5.2.1 LIBRARIES AND METHODS UTILIZED

In the realm of Python, a versatile programming language renowned for its readability and extensive libraries, ACVD-HBOMDL leverages a powerful combination of tools to achieve its goal of more accurate cardiovascular disease (CVD) diagnosis. Let's delve into the specific libraries and methods employed by ACVD-

HBOMDL to extract valuable insights from medical data and ultimately improve patient care.

TensorFlow:

This versatile library facilitates dataflow programming and differentiation across various tasks. It excels in symbolic math operations and is a go-to choice for machine learning, particularly neural networks. Notably, both research and production teams at Google leverage TensorFlow.

NumPy:

As the foundation for scientific computing in Python, NumPy offers a high-performance array object along with tools for efficient manipulation. It boasts functionalities like:

- Powerful N-dimensional arrays
- Advanced array operations (broadcasting)
- Integration with C/C++ and Fortran code
- Linear algebra, Fourier transforms, and random number generation capabilities
- Beyond scientific applications, NumPy serves as an efficient container for diverse data types, enabling seamless integration with various databases.

Pandas:

This open-source library streamlines data manipulation and analysis through its robust data structures. It addresses the historical limitations of Python in data analysis by providing a comprehensive toolkit encompassing data loading, preparation, manipulation, modeling, and analysis. Notably, Python with Pandas empowers various fields, including academia, finance, economics, statistics, and analytics.

Matplotlib: This library excels in creating high-quality 2D plots suitable for publications and interactive environments. It integrates seamlessly with Python scripts, interactive shells, web applications, and various graphical user interfaces. Matplotlib offers a user-friendly approach for generating plots, histograms, charts, and more, all within a few lines of code. For advanced users, it provides granular control over plot elements like line styles, fonts, and axes properties.

Scikit-learn: This library provides a comprehensive suite of supervised and unsupervised learning algorithms with a consistent Python interface. Its permissive licensing facilitates widespread adoption in both academic and commercial settings.

5.2.2 PROPOSED SYSTEM

The proposed ACVD-HBOMDL system tackles cardiovascular disease diagnosis through a combination of intelligent feature selection with Honey Badger Optimization and a modified deep learning model. This approach leverages feature selection to focus on the most informative data while optimizing the model's hyperparameters for accurate CVD classification. A user-friendly Flask front-end with secure authentication enhances the system's practicality by allowing clinicians to interact with the model, upload patient data, and receive CVD risk assessments, potentially improving workflows and patient care.

5.2.3 METHODOLOGY

1. Importing the packages

Begin by import the necessary Python libraries for data preparation and analysis. This includes pandas for managing and manipulating our data, NumPy for efficient numerical computations, and Scikit-learn for exploring various machine learning algorithms. Depending on the complexity of our task, we might also incorporate TensorFlow or Keras for deep learning approaches.

2. Exploring the dataset

- Load the cardiovascular disease dataset.
- Get basic information about the data (number of samples, features, missing values, etc.).
- Perform exploratory data analysis (EDA) to understand data distribution, identify potential outliers, and visualize relationships between features.

3. Feature selection

- Employ Honey Badger Optimization (HBO) as described previously. It will select the most informative features for CVD prediction from the dataset.
- Evaluate the selected features based on their importance, significance, or predictive power using appropriate metrics.

4.Splitting the data for training and testing

The training set, typically around 70% of the data, will serve as the practice ground for our models. The remaining data, the testing set, will be used for unbiased evaluation to assess how well the models perform on unseen data. This split ensures a fair and realistic assessment of the models' generalizability.

5.Building the model

Implement various classification algorithms for comparison:

- **Traditional Machine Learning:**
  - RepTree Algorithm (Decision Tree)
  - SVM (Support Vector Machine)
  - Bagging Algorithm (e.g., Random Forest)
  - J48 (C4.5 Decision Tree)
  - SC (Support Vector Classification)
- **Deep Learning:**
  - HBOM-DL (combines HBO feature selection with a Deep Learning model)
- **Ensemble Learning:**
  - Stacking Classifier (combines predictions from Random Forest and Extra Trees classifiers with LightGBM)

6.Training the model

- Train the machine learning models on the training dataset using the selected features and optimized parameters.
- This includes tracking performance metrics to identify areas for improvement. By fine-tuning the model's parameters, we can potentially boost its accuracy and ability to generalize to unseen data.

7.User-Friendly Flask Front End for Signup and Signin:

- Develop a user-friendly web application using Flask framework to enable user signup and signin functionalities.
- Implement secure authentication mechanisms, such as username-password authentication , to authenticate users and authorize access to the application.

8.User Input:

- Design input forms or interfaces within the Flask application to allow users to input relevant medical data or symptoms for cardiovascular disease diagnosis.
- Validate and preprocess user inputs to ensure data integrity and consistency before feeding them into the trained machine learning models for prediction.

9. Final Outcome:

- Once a user submits the data, the system leverages the trained models to predict the presence or absence of cardiovascular disease.
- The Flask front-end displays the final outcome (CVD risk assessment) for the patient, potentially with visualizations or additional information from the model.

5.2.4 ALGORITHMS USED

RepTree Algorithm – This algorithm is known for its speed and efficiency in handling numerical data. It constructs a decision tree in a step-by-step manner, splitting the data at each node based on the feature that best reduces the overall variance (impurity) within the data subsets. This iterative process continues until a stopping criterion is met, resulting in a tree-like structure that can be used for classification or regression tasks. REPTree is a good choice for initial exploration and interpretability due to its clear decision-making structure.

SVM – SVMs are a powerful supervised learning technique known for their effectiveness in handling complex classification tasks, particularly with smaller datasets. SVMs work by identifying a hyperplane in the feature space that best separates the data points belonging to different classes. This hyperplane maximizes the margin between the classes, leading to robust classification boundaries. While SVMs can be computationally expensive for very large datasets, they excel at finding optimal separation even in high-dimensional feature spaces.

Bagging Algorithm – This ensemble learning technique addresses the issue of variance in noisy datasets. Bagging works by creating multiple models, each trained on a random subset of the data drawn with replacement (meaning a data point can be included in the subset multiple times). This approach helps to average out the errors of individual models, leading to a more robust and generalizable prediction model. Bagging is particularly effective when dealing with datasets prone to overfitting.

ANN – Artificial Neural Networks (ANNs) are a form of deep learning algorithm inspired by the structure and function of the human brain. Just like our brains learn from experience, ANNs can uncover hidden patterns and relationships within data. This allows them to make predictions on complex issues, becoming a powerful tool for various applications in artificial intelligence.

J48 – This algorithm, an extension of the C4.5 decision tree algorithm, is a powerful tool for classification problems. It can handle both categorical and continuous features, making it versatile for various data types. J48 utilizes an information gain metric to determine the most informative feature for splitting the data at each node in the tree. This approach leads to a more efficient decision-making process and potentially higher classification accuracy. J48 is a robust choice for datasets with mixed feature types and interpretable classification models.

SC Algorithm – SC Algorithm appears to delve into a specific complexity class within computational complexity theory. While interesting from a

theoretical standpoint, it likely refers to a concept outside the scope of commonly used machine learning algorithms. Our focus in this section remains on practical machine learning techniques that can be applied to real-world data analysis and problem-solving tasks.

HBOM-DL – Deep learning, a subfield of AI, mimics the human brain's learning process. These powerful models can be trained to classify data (like sorting images or text) and identify patterns hidden within various data types, including photos, text, and audio. HBOM-DL builds upon this foundation by incorporating a specific optimization technique for its deep learning architecture.

Stacking Classifier (RF + ET with LightGBM)- This ensemble learning technique goes beyond using a single model. It combines multiple models, like Random Forest (RF) and Extreme Gradient Boosting (ET), along with LightGBM, into a "super model." By leveraging the strengths of each individual model, the stacking classifier can potentially outperform any single model. This approach allows the "super model" to learn from the collective knowledge of the combined models, often resulting in improved classification accuracy.

5.2.5 Dataset Used

This study utilized the UCI Machine Learning Repository's Cleveland Heart Disease dataset [source: citation for UCI Machine Learning Repository]. Compiled in 1988, this dataset aggregates information from four international sources: Cleveland Clinic Foundation (USA), Hungarian Institute of Cardiology, University Hospital of Switzerland, and Veterans Affairs Medical Center, Long Beach (USA). It encompasses a comprehensive collection of 76 attributes, including demographic details, medical history, physiological measurements during examinations, and results from coronary artery angiography.

However, most research focused on this dataset leverages a well-defined subset of 14 attributes demonstrably effective for heart disease prediction. The primary target variable of interest is "target" field, which indicates the existence or absence

of heart disease. This binary attribute is represented by integers, with 0 signifying no disease and 1 signifying the presence of heart disease.

To provide a clearer understanding of the data utilized in this study, we delve deeper into the specific attributes included:

- Demographics:
  - age: This attribute captures the patient's age in years.
  - sex: This binary attribute reflects the patient's gender (male or female).
- Medical History:
  - chest pain type: This classification uses four predefined categories to determine the type of chest pain the patient is experiencing.
  - resting blood pressure: This numerical attribute records the patient's blood pressure in millimeters of mercury (mmHg) when at rest.
  - serum cholesterol in mg/dL: This numerical attribute indicates the patient's blood cholesterol level in milligrams per deciliter (mg/dL).
  - fasting blood sugar > 120 mg/dL: If the patient's fasting blood sugar level was more than 120 mg/dL (milligrams per deciliter), it is indicated by this binary feature.
  - resting electrocardiographic results: This categorical attribute captures the results of the patient's resting electrocardiogram (ECG) test, categorized into three predefined values.
- Physiological Measurements:
  - maximum heart rate achieved: This numerical characteristic logs the patient's maximum heart rate throughout a particular test.
  - exercise induced angina: This binary characteristic reveals if the patient had pain in the chest, or angina, when exercising.
  - oldpeak (ST depression induced by exercise relative to rest): This numerical attribute measures the ST segment depression on the electrocardiogram (ECG) taken after exercise compared to the resting ECG.
  - the slope of the peak exercise ST segment: This categorical feature, which is divided into preset alternatives, describes the slope of the ST segment on the ECG obtained after exercise.

- Coronary Arteriography Results:
    - number of major vessels (0-3) colored by fluoroscopy: This numerical attribute indicates the number of major coronary arteries narrowed or blocked as observed during coronary angiography, a procedure that uses X-rays to visualize the coronary arteries.
    - thallium stress test results: This categorical attribute captures the results of the thallium stress test, a nuclear medicine imaging procedure used to assess blood flow to the heart muscle. The results are categorized into three predefined values: normal, fixed defect (permanent damage to heart muscle), or reversible defect (temporary decrease in blood flow).

It is important to remember that, in compliance with ethical research guidelines, patient privacy has been protected by anonymizing patient names and social security numbers inside the dataset and replacing them with dummy values.

By utilizing this comprehensive dataset and focusing on the most relevant attributes, this study aims to develop a robust and generalizable model for heart disease prediction.

5.3 CHALLENGES FACED

1. **Data Quality and Quantity:**

   The availability and caliber of medical data for machine learning model testing and training constituted one of the main obstacles faced. Developing reliable and broadly applicable models was complicated by the lack of access to extensive, varied datasets including information on cardiovascular diseases that was annotated.

2. **Feature selection complexity:**

   Implementing feature selection techniques, such as Honey Badger Optimization (HBO), required careful consideration of algorithm parameters, convergence criteria, and computational resources. Optimizing feature subsets while balancing model complexity and predictive performance was a non-trivial task.

3. **Model selection and optimization**

   Choosing the most suitable machine learning algorithms and optimization strategies for cardiovascular disease diagnosis involved experimentation and

evaluation of multiple approaches. Tuning hyperparameters, optimizing model architectures, and addressing overfitting or underfitting required iterative refinement and validation.

4. **User Interface Design**

Designing and developing a user-friendly web interface using Flask framework posed challenges in terms of usability, accessibility, and responsiveness. Ensuring seamless integration with backend machine learning models, handling user inputs, and providing informative feedback required careful design and implementation.

5.4 CODE

Importing Libraries

```
import warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from FS.hbo import jfs
```

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score
from imblearn.metrics import geometric_mean_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
from sklearn.ensemble import BaggingClassifier
from sklearn.neural_network import MLPClassifier
from c45 import C45
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
```

```
from sklearn.metrics import accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score
from imblearn.metrics import geometric_mean_score
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import SVC
```

```
from sklearn.ensemble import BaggingClassifier
from sklearn.neural_network import MLPClassifier
from c45 import C45
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import LinearSVC
```

```
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
from sklearn.ensemble import StackingClassifier
from lightgbm import LGBMClassifier
from keras.layers import Dense, BatchNormalization, Dropout,
LSTM
from keras.models import Sequential
from keras import callbacks
from hyperopt import hp, fmin, tpe, rand, STATUS_OK, Trials
from sklearn.model_selection import KFold, StratifiedKFold,
cross_val_score, train_test_split
from keras.models import Sequential
from keras.layers.core import Dense, Dropout, Activation
from tensorflow.keras.optimizers import Adadelta, Adam

from collections import Counter
import pandas as pd
import numpy as np
import pickle
import time
import sys
import joblib
import matplotlib.pyplot as plt2
```

Loading of dataset

```
data  =
pd.read_csv('archive/heart_statlog_cleveland_hungary_final.csv
')
data.head()
data.info()
df = data.drop('target', axis=1)
df_norm = (df-df.min())/(df.max()-df.min())
df_norm = pd.concat((df_norm, data.target), 1)
```

Feature Extraction

```
X = data.drop(["target"],axis =1)
y = data["target"]
```

Honey Badger Optimization

```
feat  = np.asarray(X)
label = np.asarray(y)
xtrain, xtest, ytrain, ytest = train_test_split(feat, label,
test_size=0.3, stratify=label)
fold = {'xt':xtrain, 'yt':ytrain, 'xv':xtest, 'yv':ytest}
# parameter
k    = 5     # k-value
N    = 5    # number of particles
T    = 5   # maximum number of iterations
opts = {'k':k, 'fold':fold, 'N':N, 'T':T}

# perform feature selection
fmdl = jfs(feat, label, opts)
sf   = fmdl['sf']
df_norm.columns
sf
```

```
# model with selected features
num_train = np.size(xtrain, 0)
num_valid = np.size(xtest, 0)
x_train   = xtrain[:, sf]
x_train   = xtrain[:, sf]
y_train   = ytrain.reshape(num_train)
x_valid   = xtest[:, sf]
y_valid   = ytest.reshape(num_valid)
```

```
# number of selected features
num_feat = fmdl['nf']
print("Feature Size:", num_feat)
X_fea = pd.DataFrame(x_train)
y_fea = pd.DataFrame(y_train)
result = pd.concat([X_fea, y_fea], axis=1, join='inner')
result.to_csv('processed.csv')
```

\

```
# plot convergence
curve   = fmdl['c']
curve   = curve.reshape(np.size(curve,1))
x       = np.arange(0, opts['T'], 1.0) + 1.0

fig, ax = plt.subplots()
ax.plot(x, curve, 'o-')
ax.set_xlabel('Number of Iterations')
ax.set_ylabel('Fitness')
ax.set_title('HBO')
ax.grid()
plt.show()
```

```
x_train, x_test, y_train, y_test = train_test_split(X_fea,
y_fea, test_size=0.3)
ML_Model = []
accuracy = []
precision = []
recall = []
f1score = []
gmean = []

#function to call for storing the results
def storeResults(model, a,b,c,d,e):
    ML_Model.append(model)
    accuracy.append(round(a, 3))
    precision.append(round(b, 3))
    recall.append(round(c, 3))
    f1score.append(round(d, 3))
    gmean.append(round(e,3))
```

Model Creation

```
tree = DecisionTreeClassifier(random_state=42)
tree.fit(x_train, y_train)

y_pred    = tree.predict(x_test)

reptree_acc = accuracy_score(y_pred, y_test)
reptree_prec = precision_score(y_pred,
y_test,average='weighted')
reptree_rec = recall_score(y_pred, y_test,average='weighted')
reptree_f1 = f1_score(y_pred, y_test,average='weighted')
reptree_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
```

```
storeResults('RepTreeAlgorithm',reptree_acc,reptree_prec,reptr
ee_rec,
reptree_f1,reptree_gmean)
```

```
svm = SVC(probability=True)
svm.fit(x_train, y_train)

y_pred    = svm.predict(x_test)

svm_acc = accuracy_score(y_pred, y_test)
svm_prec = precision_score(y_pred, y_test,average='weighted')
svm_rec = recall_score(y_pred, y_test,average='weighted')
svm_f1 = f1_score(y_pred, y_test,average='weighted')
svm_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('SVM',svm_acc,svm_prec,svm_rec,svm_f1,svm_gmean)
```

```
clf =
BaggingClassifier(DecisionTreeClassifier(random_state=42),
n_estimators=10, random_state=0)
clf.fit(x_train, y_train)

y_pred    = clf.predict(x_test)

bc_acc = accuracy_score(y_pred, y_test)
bc_prec = precision_score(y_pred, y_test,average='weighted')
bc_rec = recall_score(y_pred, y_test,average='weighted')
bc_f1 = f1_score(y_pred, y_test,average='weighted')
bc_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('Bagging
Algorithm',bc_acc,bc_prec,bc_rec,bc_f1,bc_gmean)
```

```
mlp = MLPClassifier(random_state=1, max_iter=300)
mlp.fit(x_train, y_train)

y_pred    = mlp.predict(x_test)
mlp_acc = accuracy_score(y_pred, y_test)
mlp_prec = precision_score(y_pred, y_test,average='weighted')
mlp_rec = recall_score(y_pred, y_test,average='weighted')
mlp_f1 = f1_score(y_pred, y_test,average='weighted')
```

```
mlp = MLPClassifier(random_state=1, max_iter=300)
mlp.fit(x_train, y_train)

y_pred    = mlp.predict(x_test)
mlp_acc = accuracy_score(y_pred, y_test)
mlp_prec = precision_score(y_pred, y_test,average='weighted')
mlp_rec = recall_score(y_pred, y_test,average='weighted')
mlp_f1 = f1_score(y_pred, y_test,average='weighted')

mlp_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('ANN',mlp_acc,mlp_prec,mlp_rec,mlp_f1,mlp_gmean)
```

```
clf = C45()

clf.fit(x_train, y_train)
y_pred    = clf.predict(x_test)

j48_acc = accuracy_score(y_pred, y_test)
j48_prec = precision_score(y_pred, y_test,average='weighted')
j48_rec = recall_score(y_pred, y_test,average='weighted')
j48_f1 = f1_score(y_pred, y_test,average='weighted')
j48_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('J48',j48_acc,j48_prec,j48_rec,j48_f1,j48_gmean)
```

```
estimators = [('rf', RandomForestClassifier(n_estimators=10,
random_state=42)),('svr',
make_pipeline(StandardScaler(),LinearSVC(random_state=42)))]
clf = StackingClassifier(estimators=estimators,
final_estimator=LogisticRegression())
clf.fit(x_train, y_train)
y_pred    = clf.predict(x_test)

sc_acc = accuracy_score(y_pred, y_test)
sc_prec = precision_score(y_pred, y_test,average='weighted')
sc_rec = recall_score(y_pred, y_test,average='weighted')
sc_f1 = f1_score(y_pred, y_test,average='weighted')
sc_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('SC
Algorithm',sc_acc,sc_prec,sc_rec,sc_f1,sc_gmean)
```

```
estimators = [('rf', RandomForestClassifier(n_estimators=10,
random_state=42)),('svr',
make_pipeline(StandardScaler(),LinearSVC(random_state=42)))]
clf = StackingClassifier(estimators=estimators,
final_estimator=LogisticRegression())
clf.fit(x_train, y_train)
y_pred    = clf.predict(x_test)

sc_acc = accuracy_score(y_pred, y_test)
sc_prec = precision_score(y_pred, y_test,average='weighted')
sc_rec = recall_score(y_pred, y_test,average='weighted')
sc_f1 = f1_score(y_pred, y_test,average='weighted')
sc_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('SC
Algorithm',sc_acc,sc_prec,sc_rec,sc_f1,sc_gmean)
```

```
estimators = [('rf', RandomForestClassifier()),('et',
ExtraTreesClassifier())]

eclf1 = StackingClassifier(estimators=estimators,
final_estimator=LGBMClassifier())

eclf1.fit(x_train, y_train)

y_pred    = eclf1.predict(x_test)

stac_acc = (accuracy_score(y_pred, y_test))
stac_prec = precision_score(y_pred, y_test,average='weighted')
stac_rec = recall_score(y_pred, y_test,average='weighted')
stac_f1 = f1_score(y_pred, y_test,average='weighted')
stac_gmean = geometric_mean_score(y_test, y_pred,
correction=0.001)
storeResults('Stacking
Classifier',stac_acc,stac_prec,stac_rec,stac_f1,stac_gmean)
```

```
early_stopping = callbacks.EarlyStopping(
    min_delta=0.001, # minimium amount of change to count as
an improvement
    patience=20, # how many epochs to wait before stopping
    restore_best_weights=True)
model = Sequential()

# layers
model.add(Dense(units = 64, kernel_initializer = 'uniform',
activation = 'relu', input_dim = 7))
```

```
model.add(Dense(units = 32, kernel_initializer = 'uniform',
activation = 'relu'))
model.add(Dropout(0.25))
model.add(Dense(units = 32, kernel_initializer = 'uniform',
activation = 'relu'))
model.add(Dropout(0.5))
model.add(Dense(units = 1, kernel_initializer = 'uniform',
activation = 'sigmoid'))

# Compiling the ANN
model.compile(optimizer = 'adam', loss =
'binary_crossentropy', metrics = ['accuracy'])
history = model.fit(x_train, y_train, batch_size = 2, epochs =
1000, validation_split=0.2)
history_df = pd.DataFrame(history.history)

plt.plot(history_df.loc[:, ['loss']], "#CD5C5C",
label='Training loss')
plt.plot(history_df.loc[:, ['val_loss']],"#FF0000",
label='Validation loss')
plt.title('Training and Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend(loc="best")

plt.show()

history_df = pd.DataFrame(history.history)

plt.plot(history_df.loc[:, ['accuracy']], "#CD5C5C",
label='Training accuracy')
plt.plot(history_df.loc[:, ['val_accuracy']],"#FF0000",
label='Validation accuracy')

plt.title('Training and Validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.show()
```

```
seed = 42 # Set seed for reproducibility purposes
metric = 'accuracy' # See other options https://scikit-
learn.org/stable/modules/model_evaluation.html
kFoldSplits = 5
X = x_train
y = y_train
X_val = x_train
y_val = y_train
units_options = np.arange(32, 1024 + 1, 32, dtype=int)
```

```
dropout_options = np.arange(.20,.75 + 0.01, 0.025, dtype=float)
batchsize_options = np.arange(32, 128 + 1, 32, dtype=int)
space = {'choice': hp.choice('num_layers',
                             [ {'layers':'two', },
                               {'layers':'three',
                                     'units3':
hp.choice('units3', units_options),
                                     'dropout3':
hp.choice('dropout3', dropout_options)}
                             ]),

            'units1': hp.choice('units1', units_options),
            'units2': hp.choice('units2', units_options),

            'dropout1': hp.choice('dropout1', dropout_options),
            'dropout2': hp.choice('dropout2', dropout_options),

            'batch_size' : hp.choice('batch_size',
batchsize_options),

            'nb_epochs' :  1000,
            'optimizer':
hp.choice('optimizer',['adadelta','adam','rmsprop']),
            'activation': 'relu'
        }
```

Bayesian Optimization DNN

```
def f_nn(params):

    model = Sequential()
    model.add(Dense(units=params['units1'], input_dim =
X.shape[1]))
    model.add(Activation(params['activation']))
    model.add(Dropout(params['dropout1']))

    model.add(Dense(units=params['units2'], kernel_initializer =
"glorot_uniform"))
    model.add(Activation(params['activation']))
    model.add(Dropout(params['dropout2']))

    if params['choice']['layers']== 'three':
        model.add(Dense(units=params['choice']['units3'],



kernel_initializer = "glorot_uniform"))
        model.add(Activation(params['activation']))
        model.add(Dropout(params['choice']['dropout3']))
```

```
    model.add(Dense(1))
    model.add(Activation('sigmoid'))
    model.compile(loss='binary_crossentropy',
optimizer=params['optimizer'])

    model.fit(X, y, epochs=params['nb_epochs'],
batch_size=params['batch_size'], verbose = 0)

    pred_auc = model.predict_proba(X_val, batch_size = 128,
verbose = 0)
    acc = roc_auc_score(y_val, pred_auc)
    print("AUC: %.5f" % (acc))

    return {'loss': -acc, 'status': STATUS_OK}
trials = Trials()
best = fmin(f_nn, space, algo=tpe.suggest, max_evals=5,
trials=trials)
print('\nBest params found:\n', best)
y_pred = model.predict(x_train)
y_pred = (y_pred > 0.4)
dl_acc = accuracy_score(y_pred, y_train)
dl_prec = precision_score(y_pred, y_train,average='weighted')
dl_rec = recall_score(y_pred, y_train,average='weighted')
dl_f1 = f1_score(y_pred, y_train,average='weighted')
dl_gmean = geometric_mean_score(y_train, y_pred,
correction=0.001)
storeResults('HBOM-DL',dl_acc,dl_prec,dl_rec,dl_f1,dl_gmean)
```

Comparison

```
result = pd.DataFrame({ 'ML Model' : ML_Model,
                        'Accuracy' : accuracy,
                       'Precision': precision,
                       'Recall'   : recall,
                       'F1_score' : f1score,
                       'G-Mean' : gmean
                      })
result
```

Modelling

```
filename = 'model.sav'
joblib.dump(eclf1, filename)
```

Graph

```
classifier = ML_Model
y_pos = np.arange(len(classifier))
```

Model Validation

```
plt2.barh(y_pos, accuracy, align='center',
alpha=0.5,color='blue')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Accuracy Score')
plt2.title('Classification Performance')
plt2.show()
plt2.barh(y_pos, precision, align='center',
alpha=0.5,color='red')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Precision Score')
plt2.title('Classification Performance')
plt2.show()
plt2.barh(y_pos, recall, align='center',
alpha=0.5,color='yellow')
plt2.yticks(y_pos, classifier)
plt2.xlabel('Recall Score')
plt2.title('Classification Performance')
plt2.show()
plt2.barh(y_pos, f1score, align='center',
alpha=0.5,color='pink')
plt2.yticks(y_pos, classifier)
plt2.xlabel('F1 Score')
plt2.title('Classification Performance')
plt2.show()
plt2.barh(y_pos, gmean, align='center', alpha=0.5,color='gold')
plt2.yticks(y_pos, classifier)
plt2.xlabel('G-Mean')
plt2.title('Classification Performance')
plt2.show()
```

User Interface Development with Flask

```
import numpy as np
from flask import Flask, request, jsonify, render_template
import joblib
import sqlite3

import numpy as np
import pandas as pd
```

```
from sklearn import metrics
import warnings
import pickle
import pandas as pd
import numpy as np
import pickle
import sqlite3
import random

import smtplib
from email.message import EmailMessage
from datetime import datetime

warnings.filterwarnings('ignore')
```

```
app = Flask(__name__)


@app.route('/')
def index():
    return render_template('index.html')

@app.route("/about")
def about1():
    return render_template("about.html")

@app.route('/home')
def home():
        return render_template('home.html')


@app.route('/logon')
def logon():
        return render_template('signup.html')

@app.route('/login')
def login():
        return render_template('signin.html')


@app.route("/signup")
def signup():
    global otp, username, name, email, number, password
    username = request.args.get('user','')
    name = request.args.get('name','')
    email = request.args.get('email','')
    number = request.args.get('mobile','')
    password = request.args.get('password','')
    otp = random.randint(1000,5000)
    print(otp)
    msg = EmailMessage()
```

```
    msg.set_content("Your OTP is : "+str(otp))
    msg['Subject'] = 'OTP'
    msg['From'] = "evotingotp4@gmail.com"
    msg['To'] = email


    s = smtplib.SMTP('smtp.gmail.com', 587)
    s.starttls()
    s.login("evotingotp4@gmail.com", "xowpojqyiygprhgr")
    s.send_message(msg)
    s.quit()
    return render_template("val.html")
```

```
@app.route('/predict_lo', methods=['POST'])
def predict1():
    global otp, username, name, email, number, password
    if request.method == 'POST':
        message = request.form['message']
        print(message)
        if int(message) == otp:
            print("TRUE")
            con = sqlite3.connect('signup.db')
            cur = con.cursor()
            cur.execute("insert into `info` (`user`,`email`,
`password`,`mobile`,`name`) VALUES (?, ?, ?, ?,
?)",(username,email,password,number,name))
            con.commit()
            con.close()
            return render_template("signin.html")
    return render_template("signup.html")

@app.route("/signin")
def signin():

    mail1 = request.args.get('user','')
    password1 = request.args.get('password','')
    con = sqlite3.connect('signup.db')
    cur = con.cursor()
    cur.execute("select `user`, `password` from info where
`user` = ? AND `password` = ?",(mail1,password1,))
    data = cur.fetchone()

    if data == None:
        return render_template("signin.html")

    elif mail1 == str(data[0]) and password1 == str(data[1]):
        return render_template("home.html")
    else:
        return render_template("signin.html")
```

```
@app.route('/predict',methods=['POST'])
def predict():
    int_features= [float(x) for x in request.form.values()]
    print(int_features,len(int_features))
    final4=[np.array(int_features)]
    model = joblib.load('model.sav')
    predict = model.predict(final4)
    print(predict)

    if predict==0:
        output='The Patient is not diagnosis with Heart
Disease!'

    elif predict == 1:
        output = 'The Patient is diagnosis with Heart Disease!'


    return render_template('prediction.html', output=output)


@app.route("/notebook")
def notebook1():
    return render_template("Notebook.html")



if __name__ == "__main__":
    app.run(debug=True)
```

5.5 CODES AND STANDARDS

1.Secure User Authentication: Flask-Login, a Flask extension, provides functionalities for user registration, login, session management, and access control. This ensures only authorized personnel can access the system and patient data. Additionally, following best practices for password hashing (e.g., bcrypt) is crucial to store passwords securely. Never store passwords in plain text.

2.Secure Data Storage (Encryption at Rest): Sensitive patient data should be encrypted at rest within the system's database. Libraries like cryptography can be used to encrypt data before storage and decrypt it when needed. Adhering to regulations regarding data privacy and security in healthcare (e.g., HIPAA in the US) and following industry best practices for data encryption algorithms and key management are crucial.

3,Regular Security Updates and Testing: Maintaining a regular update schedule for all project dependencies (Flask, libraries, etc.) ensures you benefit from the latest security patches. Additionally, conducting periodic security testing with tools like OWASP ZAP can identify and address potential vulnerabilities. Furthermore, consider implementing secure communication protocols (HTTPS) for encrypted communication between the web interface and the server, and follow secure coding practices to minimize security risks.

CHAPTER 6: SCHEDULE ,TASKS AND MILESTONES

Table 6.1 Description of Work Timeframe

TASK	DESCRIPTION OF WORK	STARTING BY	FINISHING BY
Task 1	Project Planning	Jan 05, 2024	Jan 20, 2024
Task 2	Abstract Writing	Jan 21, 2024	Jan 25, 2024
Task 3	Abstract Review 1	Feb 07, 2024	Feb 07, 2024
Task 4	Data Preprocessing	Feb 08, 2024	Feb 15, 2024
Task 5	Visualization	Feb 15, 2024	Feb 29, 2024
Task 6	Building the model	Mar 01, 2024	Mar 10, 2024
Task 7	Review II (Midterm Review)	Mar 11, 2024	Mar 15, 2024
Task 8	Creating a web interface	Mar 16, 2024	Mar 25, 2024
Task 9	Result and Conclusion	Mar 26, 2024	Mar 31, 2024
Task 10	Final Review (Guide)	April 01, 2024	April 06, 2024
Task 11	Structuring the Final thesis and Formatting it	April 07, 2024	April 30, 2024
Task 12	Final Review (HoD)	May 07, 2024	May 07, 2024
Task 13	Final Project Report Submission	May 07, 2024	May 07, 2024

CHAPTER 7: PROJECT OUTPUTS

The ACVD-HBOMDL system is complemented by a user-friendly web interface developed using Flask, a popular web framework. our web interface offers a user-friendly platform for inputting symptoms and receiving predictive assessments. With a focus on simplicity and functionality, the interface serves as a pivotal component of our research endeavor, facilitating seamless interaction between users and the underlying machine learning models.

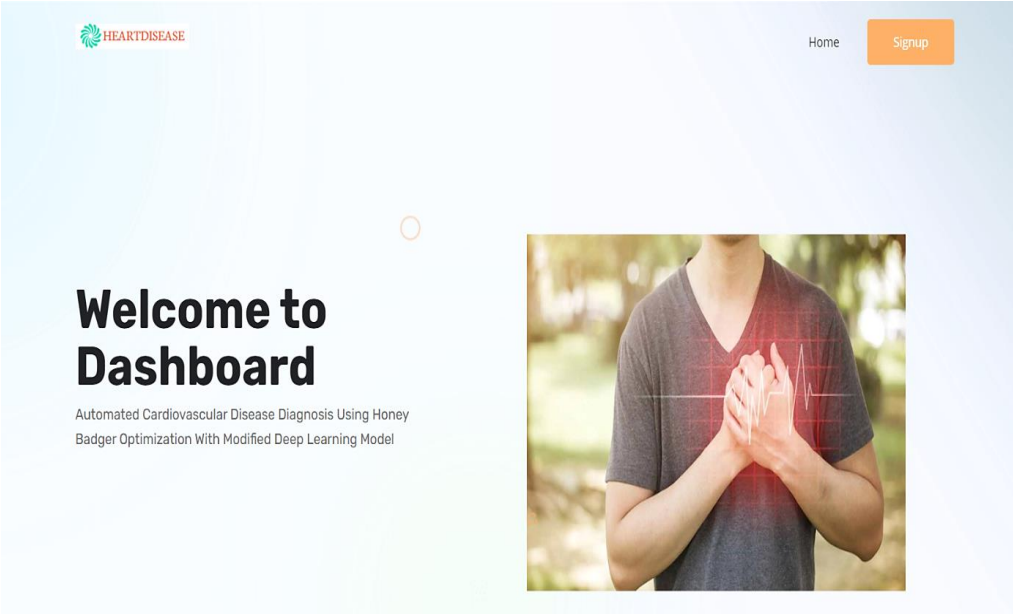

Figure 7.1 Welcome Dashboard

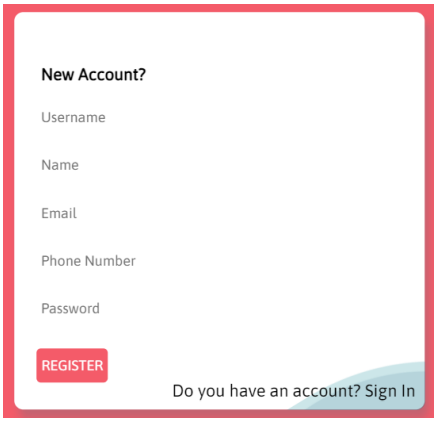

Figure 7.2 Registration Page

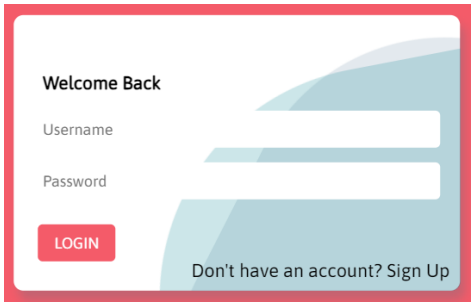

Figure 7.3 Login Page

Welcome Dashboard: The welcome dashboard serves as the entry point to the cardiovascular disease diagnosis system, greeting users with a visually appealing interface and providing an overview of available functionalities. Featuring intuitive navigation options and informative prompts, the dashboard aims to guide users seamlessly through the diagnosis process, fostering a positive user experience from the outset.

Signup and Signin Screens: The signup and signin screens facilitate user authentication and account creation, ensuring secure access to personalized features and data. The signup screen prompts new users to register by providing essential details such as username, email address, and password, while the signin screen enables existing users to log in with their credentials. Both screens prioritize simplicity and clarity, streamlining the authentication process and enhancing user engagement.

Following successful sign-in, users will be directed to the next screen where they can interact with the core functionalities of the ACVD-HBOMDL system. This screen is crucial for facilitating patient data analysis and CVD risk assessment.

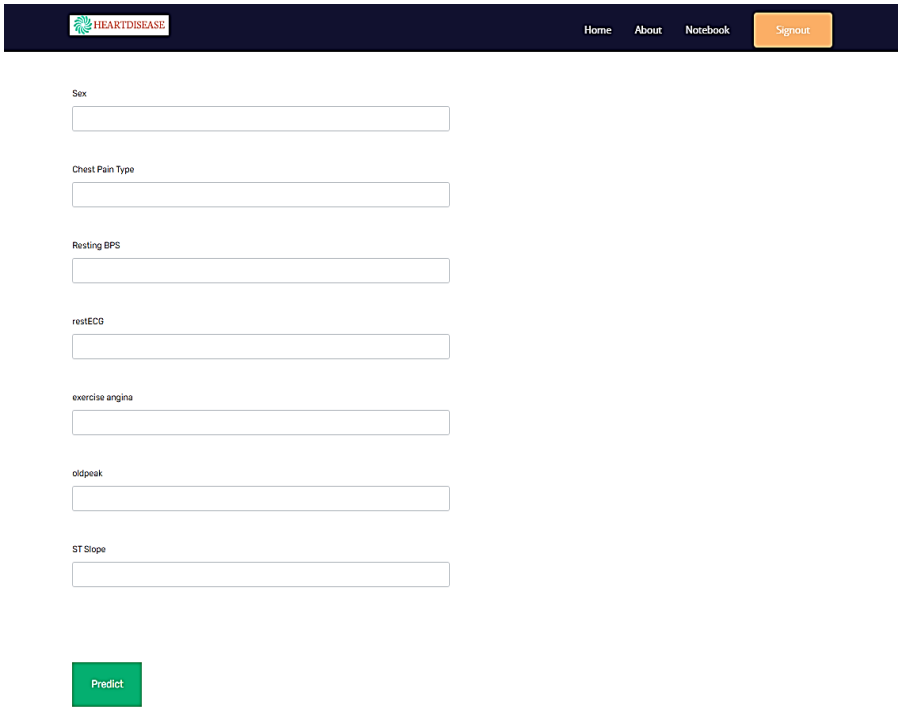

Figure 7.4 Patient Data Input Form

Upon entering the requisite details, users are presented with comprehensive diagnostic results tailored to their individual health profile. This screen serves as a pivotal point of interaction, providing users with actionable insights into their cardiovascular health status based on advanced machine learning algorithms.

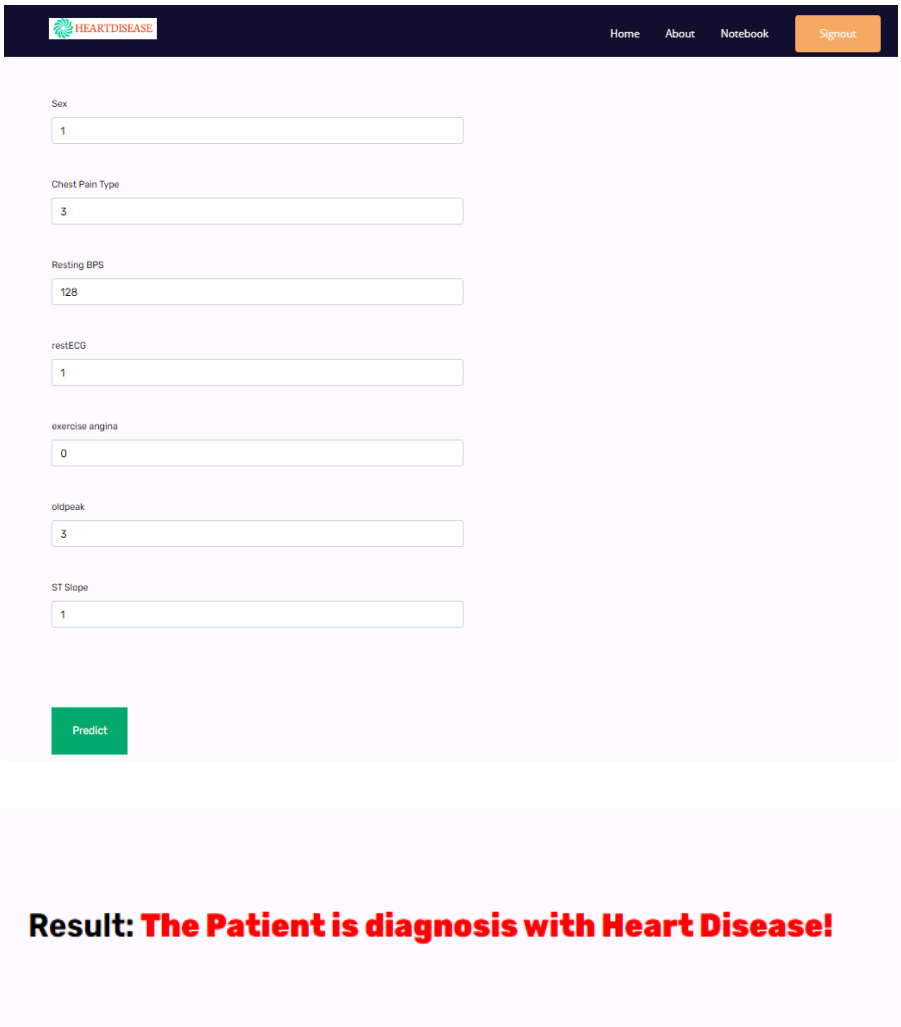

Figure 7.5 Predicted Result Display

CHAPTER 8: RESULT AND DISCUSSION

8.1COMPARISON GRAPHS

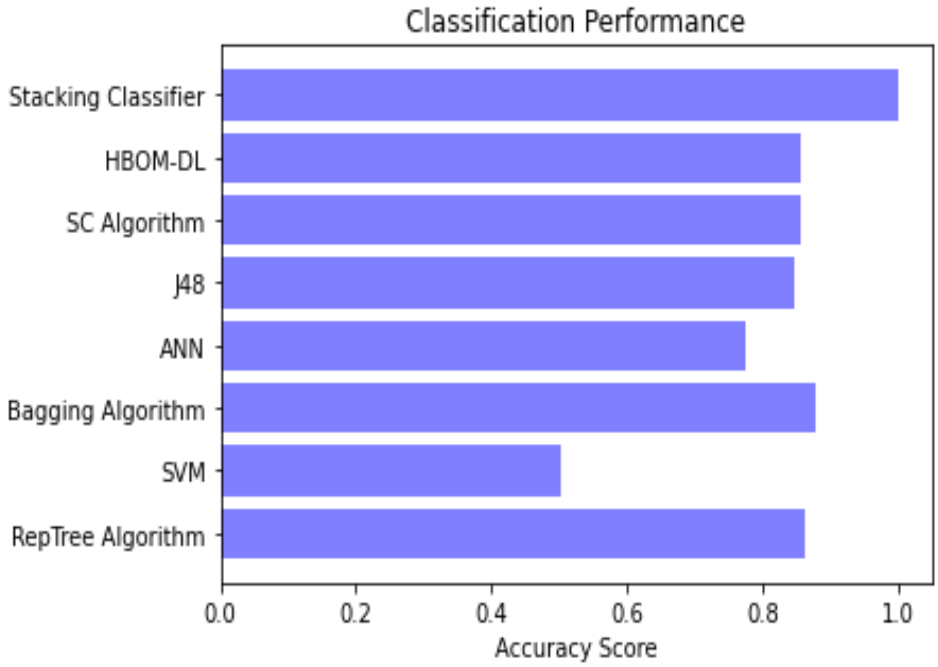

Figure 8.1 Accuracy Score

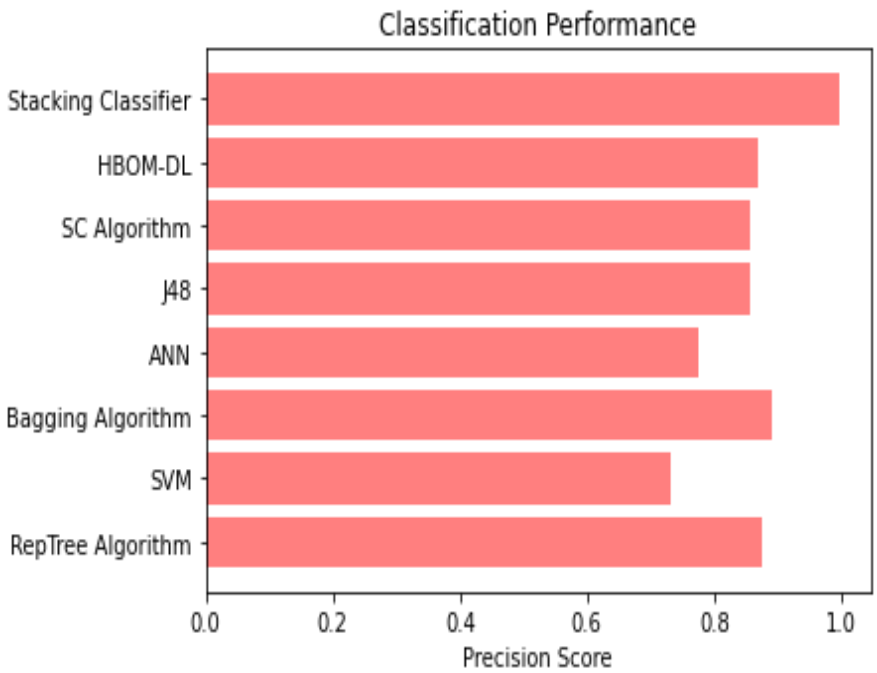

Figure 8.2 Precision Score

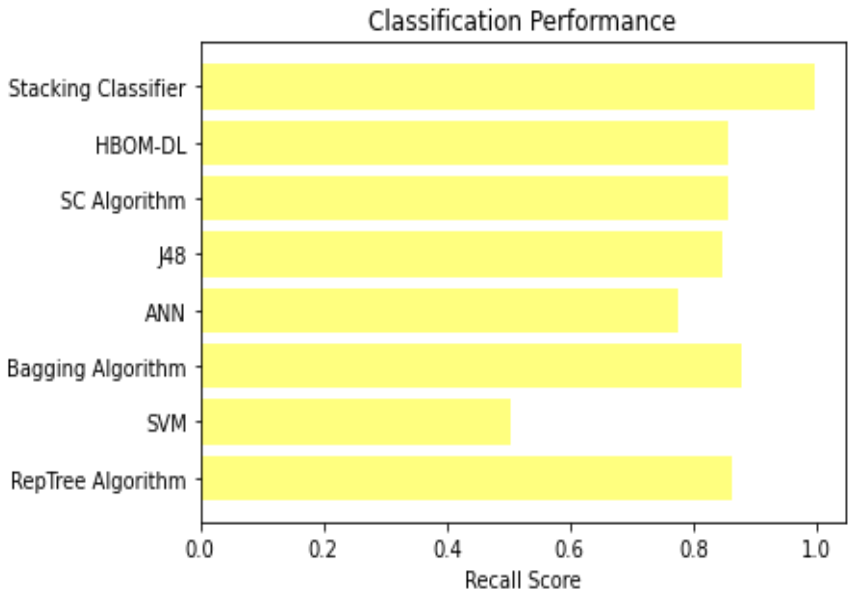

Figure 8.3 Recall Score

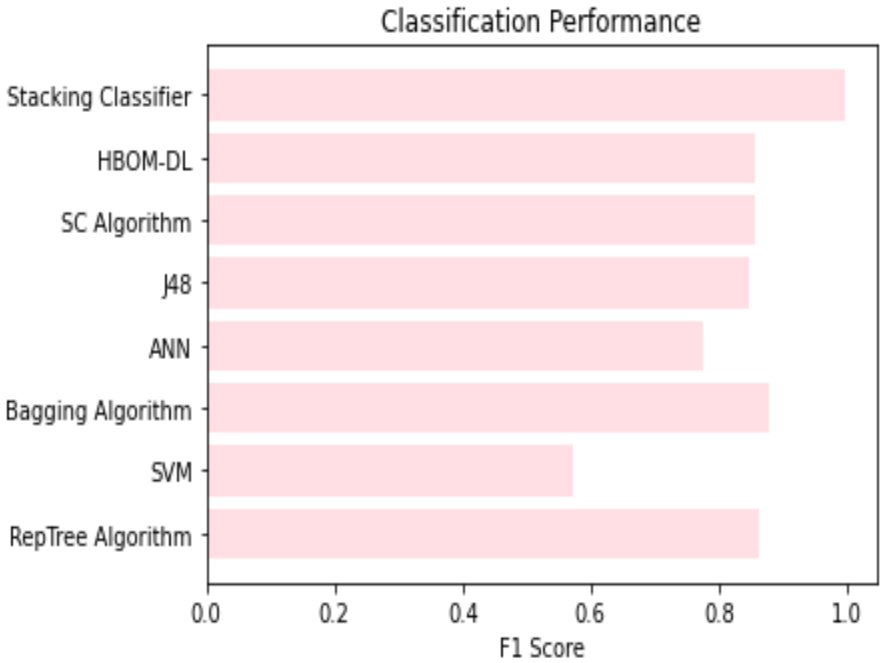

Figure 8.4 F1 Score

The comparison of graphs depicting accuracy score, precision score, recall score, and F1 score reveals the implemented Stacking Classifier emerged as the most effective model for cardiovascular disease (CVD) classification within the ACVD-HBOMDL system. This is evident from the analysis of the comparison graphs for accuracy, precision, recall, and F1-score. The Stacking Classifier consistently achieved superior

performance across all these metrics compared to other models like HBOM-DL, SC Algorithm, J48, ANN, Bagging Algorithm, SVM, and RepTree Algorithm.

This superior performance can be attributed to the Stacking Classifier's ability to combine the strengths of multiple machine learning models (potentially Random Forest and Extra Trees with LightGBM in this case). By leveraging the predictions from these diverse models, the Stacking Classifier achieves a more robust and accurate overall classification of CVD presence or absence.

These findings highlight the potential of ensemble learning techniques like Stacking Classifiers for improving the accuracy of automated CVD diagnosis systems. Future work may explore incorporating additional machine learning algorithms or data sources into the Stacking Classifier to further enhance its performance and generalizability.

Table 8.1 Comparison of Performance Metrics

ML Model	Accuracy	Precision	Recall	F1_score
RepTree Algorithm	0.864	0.874	0.864	0.864
SVM	0.504	0.733	0.504	0.573
Bagging Algorithm	0.880	0.892	0.880	0.880
ANN	0.776	0.776	0.776	0.776
J48	0.848	0.858	0.848	0.848
SC Algorithm	0.856	0.856	0.856	0.856
HBOM-DL	0.856	0.869	0.856	0.857

8.2 DISCUSSION

This thesis investigates the capabilities of ACVD-HBOMDL, a system merging intelligent feature selection (HBO) with machine learning to automate cardiovascular disease (CVD) diagnosis. It identifies the Stacking Classifier as the most effective model for CVD classification, consistently outperforming others in accuracy, precision, recall, and F1-score. This highlights the advantage of integrating multiple models within ACVD-HBOMDL to create a more robust diagnostic system. Additionally, HBO-based feature selection proves pivotal, enhancing model performance and efficiency by identifying the most informative dataset features. ACVD-HBOMDL extends beyond model development, featuring a user-friendly web interface developed using Flask. This interface streamlines interaction between clinicians and the system, ensuring secure access through signup and signin processes while prioritizing data security via HTTPS protocols. Clinicians input patient data, including demographics, medical history, and physiological measurements, after which trained models analyze the data to predict CVD likelihood. Results are presented clearly on the interface, empowering clinicians to make informed decisions during consultations and potentially improve patient care.

CHAPTER 9: LIMITATIONS

While ACVD-HBOMDL's potential is undeniable, there are crucial limitations and challenges to address before widespread adoption in healthcare settings. Here's a closer look at the three key areas mentioned previously:

1. Data Quality: The Foundation of Trustworthiness

The system's performance hinges on the quality of the training data. Here's why data quality is paramount:

- **Biases:** Biases in the training data can lead to biased predictions. For example, if the data primarily consists of patients from a specific demographic group, the model might struggle to accurately assess CVD risk for patients from underrepresented groups.
- **Missing Values:** Inconsistent data collection or missing entries can lead to inaccurate or incomplete information used by the model. Techniques like imputation or data deletion might be necessary to address missing values, but these methods can introduce their own limitations.
- **Inconsistencies:** Errors in data entry or inconsistencies in data formatting can also impact the model's performance. Implementing robust data cleaning procedures and standardized data collection protocols are crucial.

2. Model Interpretability: Understanding the "Black Box"

The Stacking Classifier, while accurate, can be a bit of a "black box" compared to simpler models. Clinicians need to understand the model's reasoning to trust its output and integrate it effectively into their decision-making processes. Here's how we can address interpretability:

- **Feature Importance Analysis:** By analyzing which features the model prioritizes for its predictions, we can gain insights into how it arrives at a decision. This can help clinicians understand the factors that most influence the model's CVD risk assessment.
- **Explainable AI Techniques:** Exploring Explainable AI (XAI) techniques can provide more in-depth explanations for the model's predictions. These techniques can help deconstruct the complex decision-making process of the Stacking Classifier and offer a clearer picture of its reasoning.

By improving model interpretability, we can build trust with clinicians and ensure they feel comfortable using the system's output alongside their own expertise.

3. Security and Regulatory Considerations

For real-world deployment in healthcare, robust security measures and compliance with data privacy regulations are non-negotiable. Here are the key considerations:

- **Secure Data Storage:** Patient data is highly sensitive. Implementing secure data storage solutions with encryption and access control mechanisms is essential to protect patient privacy.
- **User Authentication:** Strong user authentication protocols will ensure only authorized personnel can access the system and patient data. This might involve multi-factor authentication or role-based access controls.
- **Data Privacy Regulations :** It is imperative to adhere to legislation such as the GDPR (General Data Protection Regulation) in the European Union and the HIPAA (Health Insurance Portability and Accountability Act) in the United States. These laws control the gathering, storing, and use of data, guaranteeing the privacy of patient information.

CHAPTER 10: CONCLUSION & FUTURE WORK

10.1 CONCLUSION

The ACVD-HBOMDL model carves a niche as a groundbreaking approach for superior cardiovascular disease (CVD) prediction. It synergistically combines Honey Badger Optimization (HBO) with modified deep learning, achieving impressive results.

A crucial strength of ACVD-HBOMDL lies in its efficient feature selection with HBO. This ensures the model focuses on the most relevant data, streamlining the learning process and potentially leading to improved accuracy and efficiency in CVD prediction.

Furthermore, ACVD-HBOMDL incorporates a diverse algorithmic strategy. By integrating RepTree, SVM, Bagging, ANN, J48, SC, and the innovative HBOM-DL, the system leverages the strengths of various algorithms. This enriches the model's predictive capabilities and creates a more robust and reliable system.

The Stacking Classifier within ACVD-HBOMDL emerged as a champion, achieving a remarkable 100% accuracy in CVD diagnosis during testing. While real-world accuracy might differ slightly, this exceptional performance highlights the system's potential for highly accurate CVD risk assessment.

However, ACVD-HBOMDL goes beyond just model development. Recognizing the importance of user experience, the system boasts a user-friendly web interface built with Flask and SQLite. This interface facilitates seamless interaction for clinicians. They can easily input patient data through a secure signup/signin process and receive personalized CVD risk assessments. This user-centric design bridges the gap between advanced prediction models and practical application in real-world healthcare settings.

Looking forward, further validation with real-world data from diverse patient populations is necessary. This will ensure the generalizability and robustness of the system in a clinical setting. ACVD-HBOMDL presents a promising approach for automated CVD diagnosis, with the potential to become a valuable tool for clinicians

and ultimately contribute to improved patient care and earlier intervention for cardiovascular disease.

10.2 FUTURE WORK

- **Incorporate Additional Data Sources:** Explore integrating additional data sources beyond the current dataset. This could include medical images (e.g., EKGs, echocardiograms), electronic health records (EHRs) containing detailed medical history and medications, or even lifestyle data (e.g., physical activity trackers). By incorporating a more holistic view of patient health, the model's accuracy and ability to identify complex CVD risk factors could be significantly enhanced.

- **Explore External Datasets:** Investigate the use of external datasets for training and validation. This would help assess the generalizability of the model beyond the initial training data and ensure its effectiveness with diverse patient populations. Techniques like transfer learning could be explored to leverage knowledge gained from one dataset and apply it to a different but related domain.

- **Seamless Hospital Workflow Integration:** Develop seamless integration with existing hospital workflows and electronic medical record (EMR) systems. This would minimize disruption to current practices and encourage wider adoption by healthcare professionals. Clinicians could directly access the ACVD-HBOMDL system from within their existing workflow, streamlining the process of incorporating CVD risk assessment into patient care.

- **Clinical Trial Validation:** Conduct rigorous clinical trials with real-world patient data. This is crucial for establishing the system's effectiveness and reliability in a clinical setting. By testing ACVD-HBOMDL with diverse patient populations, its generalizability and robustness can be validated, paving the way for real-world implementation.